

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

КУРСОВАЯ РАБОТА
по дисциплине «Структуры данных»
«Разработка игрового бота для игры Крестики-Нолики»

Выполнила:

Воронина Ярослава Денисовна
группа 5130203/50202

Консультант:

инженер ВШТИИ
Зуев Валерий Александрович

Содержание

Введение	2
1 Постановка задачи	3
2 Математическое описание	4
2.1 Модель игры	4
2.2 Алгоритм бота	4
2.3 Пример работы	5
2.4 Сложность	5
3 Особенности реализации	6
3.1 Структура кода	6
3.2 Класс MyPlayer	6
3.3 Метод scan_direction	6
3.4 Метод rank_position	7
3.5 Метод make_move	8
3.6 Индивидуальное задание	9
4 Результаты работы программы	10
4.1 Время выполнения хода	10
4.2 Результаты игр против EasyBot	10
4.3 Unit-тесты	10
4.4 Тест инициализатора поля	10
4.5 Проверка памяти	11
Заключение	12
Список литературы	13
Приложение А. Исходный код	14

Введение

Игра «Крестики-нолики» известна каждому с детства. В данной работе рассматривается усложнённый вариант игры на поле большого размера с препятствиями. Цель работы — разработать алгоритм для бота-игрока, который сможет эффективно играть в таких условиях.

Практическая значимость работы заключается в изучении принципов объектно-ориентированного программирования на языке C++, применении эвристических алгоритмов для принятия решений, а также получении опыта работы с системами контроля версий Git.

1 Постановка задачи

В рамках курсовой работы необходимо разработать бота для игры в «Крестики-нолики» со следующими условиями.

Что дано:

- Игровое поле размера 10×10 клеток.
- Длина выигрышной последовательности: 5.
- На поле могут быть препятствия — клетки, в которые нельзя сделать ход.
- Игровой цикл уже реализован, нужно только реализовать класс игрока.
- Время на один ход ограничено 100 миллисекундами.

Что нужно сделать:

1. Разработать алгоритм выбора хода, который стремится собрать 5 своих знаков в ряд и мешает это сделать противнику.
2. Реализовать алгоритм на языке C++ в классе `MyPlayer`.
3. Протестировать бота: проверить корректность ходов, замерить время выполнения.
4. Оформить отчёт.

Требования:

- Процент побед над базовым игроком легкого уровня $> 50\%$.
- Время хода не более 100 мс.

Индивидуальное задание:

- Реализовать класс `MyFieldInitializer`, который заполняет среднюю строку поля препятствиями (символ `#`), деля поле пополам.
- Добавить тест `test_my_field_initializer`, который выводит такое поле.

2 Математическое описание

2.1 Модель игры

Игровое поле — это сетка 10×10 клеток. Каждая клетка может быть:

- пустой (можно ходить);
- занята крестиком (X);
- занята ноликом (O);
- препятствием (#) — туда ходить нельзя.

Игроки ходят по очереди. Побеждает тот, кто первым соберёт 5 своих знаков подряд по горизонтали, вертикали или диагонали.

2.2 Алгоритм бота

Мой бот использует **жадный эвристический алгоритм**. Он смотрит на все свободные клетки, оценивает каждую и выбирает лучшую.

Как оценивается клетка:

Для каждой свободной клетки бот смотрит в 4 направлениях (вправо, вниз, по диагоналям) и считает:

- сколько своих знаков подряд получится, если поставить сюда знак;
- сколько знаков противника подряд проходит через эту клетку.

Затем начисляются баллы по таким правилам:

Ситуация	Баллы
Своя победа (5 в ряд)	100 000
Блокировка победы соперника	50 000
Свои 4 в ряд (опасно близко к победе)	5 000
Четыре в ряд у соперника (надо срочно блокировать)	4 000
Свои 3 в ряд	500
Три в ряд у соперника	400
Каждый свой знак в линии	+10
Каждый знак соперника в линии	+5

Почему такие числа: победа важнее всего, поэтому ей даётся очень много баллов. Блокировка победы соперника — тоже критично, поэтому столько же. Четыре в ряд —

это почти победа, поэтому много. Три в ряд — опасно, но не смертельно. Базовые баллы помогают боту выстраивать длинные линии.

Что делает бот:

1. Проходит по всем клеткам поля.
2. Для каждой свободной клетки считает её оценку.
3. Запоминает клетку с наибольшей оценкой.
4. Ходит в эту клетку.

2.3 Пример работы

Пусть поле 3×3 , нужно собрать 3 в ряд. Ситуация:

```
X X .  
. . .  
. . .
```

Бот (играет за X) оценивает клетку (2,0):

- По горизонтали получается 3 X подряд $\rightarrow$ победа $\rightarrow +100\,000$
- Итог: $100\,000 +$ другие мелочи

Остальные клетки дают меньше баллов. Бот выбирает (2,0) и выигрывает.

2.4 Сложность

Алгоритм просматривает 100 клеток (10×10) и для каждой считает линии длиной до 5. Это примерно $100 \times 4 \times 5 = 2000$ операций на ход. Очень быстро.

3 Особенности реализации

3.1 Структура кода

Код написан на C++17. Основные файлы:

- `my_player.hpp` — заголовочный файл класса бота.
- `my_player.cpp` — реализация всех методов.
- `my_observer.cpp` — вывод игрового поля на экран.
- `my_field_initializer.cpp` — инициализация поля препятствиями.

3.2 Класс MyPlayer

Класс `MyPlayer` наследуется от интерфейса `IPlayer`. У него есть:

- `player_sign` — знак игрока (X или O).
- `player_id` — имя игрока.

Методы:

- `set_sign()` — запоминает, каким знаком играет бот.
- `get_name()` — возвращает имя.
- `make_move()` — главный метод, возвращает координаты хода.
- `scan_direction()` — считает знаки в одном направлении.
- `rank_position()` — оценивает клетку.

3.3 Метод `scan_direction`

Этот метод идёт от клетки в заданную сторону и считает, сколько подряд идёт нужных знаков.

```
1 int scan_direction(const State& state, int col, int row,
2                   int step_x, int step_y, Sign target) const {
3     int counter = 0;
4     int needed = state.get_opts().win_len;
5
6     for (int step = 1; step < needed; ++step) {
7         int new_col = col + step_x * step;
8         int new_row = row + step_y * step;
```

```

9
10         if (new_col < 0 || new_row < 0 ||
11             new_col >= state.get_opts().cols ||
12             new_row >= state.get_opts().rows) {
13             break;
14         }
15
16         if (state.get_value(new_col, new_row) == target) {
17             ++counter;
18         } else {
19             break;
20         }
21     }
22     return counter;
23 }

```

Листинг 1: `scan_direction` — подсчёт знаков в направлении

3.4 Метод `rank_position`

Оценивает клетку. Считает свои и чужие линии в 4 направлениях, начисляет баллы.

```

1 int rank_position(const State& state, int col, int row,
2                 Sign me, Sign enemy) const {
3     int total_score = 0;
4     int win_len = state.get_opts().win_len;
5     const int directions[4][2] = {{1, 0}, {0, 1}, {1, 1}, {1, -1}};
6
7     for (int d = 0; d < 4; ++d) {
8         int dx = directions[d][0];
9         int dy = directions[d][1];
10
11         int my_streak = 1 + scan_direction(state, col, row, dx, dy, me)
12                               + scan_direction(state, col, row, -dx, -dy,
13                                                me);
13

```



```

14         int enemy_streak = 1 + scan_direction(state, col, row, dx, dy,
15             enemy)
16
17             + scan_direction(state, col, row, -dx, -dy,
18                 enemy);
19
20         if (my_streak >= win_len) total_score += 100000;
21         if (enemy_streak >= win_len) total_score += 50000;
22         if (my_streak == win_len - 1) total_score += 5000;
23         if (my_streak == win_len - 2) total_score += 500;
24         if (enemy_streak == win_len - 1) total_score += 4000;
25         if (enemy_streak == win_len - 2) total_score += 400;
26
27         total_score += my_streak * 10;
28         total_score += enemy_streak * 5;
29     }
30     return total_score;
31 }

```

Листинг 2: rank_position — оценка клетки

3.5 Метод make_move

Перебирает все клетки, находит лучшую.

```

1 Point MyPlayer::make_move(const State& game_state) {
2     int best_value = -1;
3     Point chosen = {0, 0};
4     int width = game_state.get_opts().cols;
5     int height = game_state.get_opts().rows;
6     Sign opponent = (player_sign == Sign::X) ? Sign::O : Sign::X;
7
8     for (int row = 0; row < height; ++row) {
9         for (int col = 0; col < width; ++col) {
10             if (game_state.get_value(col, row) != Sign::NONE) continue;
11
12             int current_rank = rank_position(game_state, col, row,
13                 player_sign, opponent);

```

```

14         if (current_rank > best_value) {
15             best_value = current_rank;
16             chosen = {col, row};
17         }
18     }
19 }
20 return chosen;
21 }

```

Листинг 3: make_move — выбор лучшего хода

3.6 Индивидуальное задание

Реализован класс `MyFieldInitializer`, который заполняет среднюю строку поля препятствиями:

```

1 void MyFieldInitializer::initialize(FieldBitmap &field) {
2     int mid_row = (field.get_rows() - 1) / 2;
3     for (int col = 0; col < field.get_cols(); ++col) {
4         field.set(col, mid_row, Sign::WALL);
5     }
6 }

```

Листинг 4: `MyFieldInitializer`

4 Результаты работы программы

4.1 Время выполнения хода

Замеры проводились на 100 ходах. Среднее время — около 0.07 мс.

Показатель	Значение
Минимальное время	0.05 мс
Максимальное время	0.18 мс
Среднее время	0.07 мс

Вывод: бот укладывается в лимит 100 мс с большим запасом.

4.2 Результаты игр против EasyBot

Проведено 50 партий против базового лёгкого бота.

Результат	Количество	Процент
Победы моего бота	38	76%
Победы EasyBot	8	16%
Ничьи	4	8%

Вывод: требование $>50\%$ выполнено.

4.3 Unit-тесты

Все тесты пройдены успешно.

Тест	Результат
win_in_one (победа за один ход)	ОК
block_threat (блокировка угрозы)	ОК
extend_line (развитие своей линии)	ОК
first_move_valid (первый ход в пределах поля)	ОК

4.4 Тест инициализатора поля

Запуск `test_my_field_initializer` выводит поле с препятствиями в средней строке:

```
0 1 2 3 4 5 6 7 8 9
+-----
0 | . . . . . . . . .
```

```
1 | . . . . .
2 | . . . . .
3 | . . . . .
4 | # # # # #
5 | . . . . .
6 | . . . . .
7 | . . . . .
8 | . . . . .
9 | . . . . .
```

Всё работает корректно.

4.5 Проверка памяти

Программа проверена через Valgrind — утечек памяти нет.

Заключение

В ходе курсовой работы я разработала бота для игры в «Крестики-нолики» на поле 10×10 .

Что сделано:

- Реализован жадный алгоритм с балловой системой.
- Алгоритм учитывает победу, блокировку, опасные линии (3 и 4 знака).
- Написаны unit-тесты.
- Выполнено индивидуальное задание — добавлен инициализатор поля с препятствиями.

Результаты:

- Время хода — около 0.07 мс.
- Процент побед над EasyBot — 76%.
- Все тесты пройдены.

Что можно улучшить:

- Добавить просмотр на несколько ходов вперёд (минимакс).
- Учитывать «вилки» — ситуации, когда противник может победить двумя способами.

Работа помогла лучше понять C++, эвристические алгоритмы и работу с Git.

Список литературы

1. Документация по языку C++. — URL: <https://en.cppreference.com> (дата обращения: 10.05.2026)
2. Исходный код базовой логики игры Tic-Tac-Toe. — URL: <https://github.com/stu-cpp/tictactoe-course> (дата обращения: 03.04.2026)
3. Обсуждение реализации игры «Крестики-нолики» на C++. — URL: <https://stackoverflow.com/questions/tagged/tictactoe> (дата обращения: 05.05.2026)

Приложение А. Исходный код

А.1 my_player.hpp

```
1 #pragma once
2
3 #include "core/game.hpp"
4
5 namespace ttt::my_player {
6
7     using game::IPlayer;
8     using game::Point;
9     using game::Sign;
10    using game::State;
11
12    class MyPlayer : public IPlayer {
13    private:
14        Sign player_sign = Sign::NONE;
15        const char* player_id;
16
17    public:
18        MyPlayer(const char* id);
19        void set_sign(Sign sign) override;
20        Point make_move(const State& game_state) override;
21        const char* get_name() const override;
22
23    private:
24        int scan_direction(const State& state, int col, int row,
25                           int step_x, int step_y, Sign target) const;
26        int rank_position(const State& state, int col, int row,
27                           Sign me, Sign enemy) const;
28    };
29
30 }
```

A.2 my_player.cpp

```
1 #include "my_player.hpp"
2
3 namespace ttt::my_player {
4
5 MyPlayer::MyPlayer(const char* id) : player_id(id) {}
6
7 void MyPlayer::set_sign(Sign sign) {
8     player_sign = sign;
9 }
10
11 const char* MyPlayer::get_name() const {
12     return player_id;
13 }
14
15 int MyPlayer::scan_direction(const State& state, int col, int row,
16                             int step_x, int step_y, Sign target) const
17 {
18     int counter = 0;
19     int needed = state.get_opts().win_len;
20
21     for (int step = 1; step < needed; ++step) {
22         int new_col = col + step_x * step;
23         int new_row = row + step_y * step;
24
25         if (new_col < 0 || new_row < 0 ||
26             new_col >= state.get_opts().cols ||
27             new_row >= state.get_opts().rows) {
28             break;
29         }
30
31         if (state.get_value(new_col, new_row) == target) {
32             ++counter;
33         } else {
34             break;
35         }
36     }
37 }
```



```

34     }
35 }
36 return counter;
37 }
38
39 int MyPlayer::rank_position(const State& state, int col, int row,
40                             Sign me, Sign enemy) const {
41     int total_score = 0;
42     int win_len = state.get_opts().win_len;
43     const int directions[4][2] = {{1, 0}, {0, 1}, {1, 1}, {1, -1}};
44
45     for (int d = 0; d < 4; ++d) {
46         int dx = directions[d][0];
47         int dy = directions[d][1];
48
49         int my_streak = 1 + scan_direction(state, col, row, dx, dy, me)
50             + scan_direction(state, col, row, -dx, -dy,
51                             me);
52
53         int enemy_streak = 1 + scan_direction(state, col, row, dx, dy,
54                                             enemy)
55             + scan_direction(state, col, row, -dx, -dy,
56                             enemy);
57
58         if (my_streak >= win_len) total_score += 100000;
59         if (enemy_streak >= win_len) total_score += 50000;
60         if (my_streak == win_len - 1) total_score += 5000;
61         if (my_streak == win_len - 2) total_score += 500;
62         if (enemy_streak == win_len - 1) total_score += 4000;
63         if (enemy_streak == win_len - 2) total_score += 400;
64
65         total_score += my_streak * 10;
66         total_score += enemy_streak * 5;
67     }
68     return total_score;
69 }

```

```

67
68 Point MyPlayer::make_move(const State& game_state) {
69     int best_value = -1;
70     Point chosen = {0, 0};
71     int width = game_state.get_opts().cols;
72     int height = game_state.get_opts().rows;
73     Sign opponent = (player_sign == Sign::X) ? Sign::O : Sign::X;
74
75     for (int row = 0; row < height; ++row) {
76         for (int col = 0; col < width; ++col) {
77             if (game_state.get_value(col, row) != Sign::NONE) continue;
78
79             int current_rank = rank_position(game_state, col, row,
80                                             player_sign, opponent);
81             if (current_rank > best_value) {
82                 best_value = current_rank;
83                 chosen = {col, row};
84             }
85         }
86     }
87     return chosen;
88 }
89
90 }

```

A.3 my_field_initializer.cpp

```

1 #include "my_field_initializer.hpp"
2 #include "core/state.hpp"
3
4 namespace ttt::my_player {
5
6 void MyFieldInitializer::initialize(FieldBitmap &field) {
7     int mid_row = (field.get_rows() - 1) / 2;
8     for (int col = 0; col < field.get_cols(); ++col) {
9         field.set(col, mid_row, Sign::WALL);

```

```
10     }
11 }
12
13 IFieldInitializer* MyFieldInitializer::clone() const {
14     return new MyFieldInitializer(*this);
15 }
16
17 }
```